Orquestração e Persistência de Dados em PHP

Padrões técnicos de serialização, tipagem rigorosa e arquitetura de projetos

NotebookLM

Roteiro Técnico

01. Exportando Estruturas para Arquivos
02. Importando Dados Externos para a Aplicação
03. Blindando a Criação de Estruturas de Dados
04. Mapeando Argumentos de Forma Explícita
05. Isolando Responsabilidades na Arquitetura

NotebookLM

O Dilema e a Ferramenta

Exportando Estruturas para Arquivos

Dados mantidos em variáveis na memória desaparecem assim que a execução do script termina.

Como podemos converter informações **estruturadas** (arrays) para um **formato universal** e **salvá-las permanentemente no disco físico**?

Solução: Combinação de serialização para um formato padrão universal (JSON) e injeção direta de conteúdo em sistema de arquivos.

Terminal

```
$string_serializada =
json_encode($array_origem);

file_put_contents('caminho/do/arquivo.ext',
$string_serializada);
```

NotebookLM

A Aplicação Prática

Exportando Estruturas para Arquivos

Contexto: Preparando o array do filme e escrevendo seu estado no diretório atual da aplicação.

```
1  // Transformando o array associativo do filme em JSON
2  $filmeComoStringJson = json_encode($filme);
3
4  // Injetando a string diretamente em um arquivo físico
5  file_put_contents(__DIR__ . '/filme.json', $filmeComoStringJson);
```

NotebookLM

O Dilema e a Ferramenta

Importando Dados Externos para a Aplicação

Arquivos de texto armazenados no disco são apenas blocos opacos de strings. Como extrair essa string de um arquivo e convertê-la de volta para uma estrutura associativa nativa que a linguagem possa manipular?

Solução: Captura do conteúdo bruto do arquivo combinada à desserialização do JSON, forçando o retorno como um array associativo.

```
$conteudo_bruto =
file_get_contents('caminho/do/arquivo.ext');

$array_manipulavel =
json_decode($conteudo_bruto, true);
```

NotebookLM

A Aplicação Prática

Importando Dados Externos para a Aplicação

Contexto: Lendo o estado persistido e recriando o catálogo de dados nativos na memória.

```php
// Mapeando a localização física
$caminhoArquivo = __DIR__ . '/filme.json';
// Extraindo o texto opaco
$conteudoArquivoFilme = file_get_contents($caminhoArquivo);
// Reconstruindo a estrutura associativa nativa
$filme = json_decode($conteudoArquivoFilme, true);
var_dump($filme);
```

NotebookLM

O Dilema e a Ferramenta

Blindando a Criação de Estruturas de Dados

Arrays associativos instanciados de forma livre dependem inteiramente da memória do desenvolvedor. A omissão de dados cruciais ou erros de digitação nas chaves quebram contratos e geram inconsistências graves no sistema.

Solução: Funções construtoras com assinatura estrita. Definimos parâmetros obrigatórios rigorosamente tipados e forçamos o tipo de retorno da função.

```
function nomeConstrutor(tipoA $param1, tipoB
$param2): tipoRetorno
{
    return [
        'chave1' => $param1,
        'chave2' => $param2
    ];
}
```

NotebookLM

A Aplicação Prática

Blindando a Criação de Estruturas de Dados

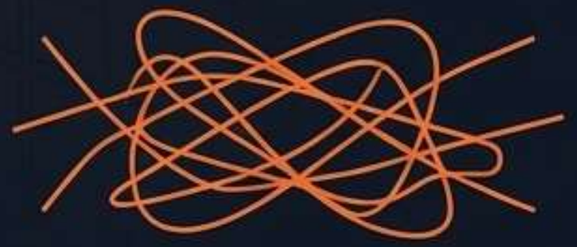
Contexto: Eliminando a criação solta e estabelecendo a fábrica tipada de filmes.

```php
function criaFilme(string $nome, int $anoLancamento, float $nota, string $genero): array
{
    return [
        'nome' => $nome,
        'ano' => $anoLancamento,
        'nota' => $nota,
        'genero' => $genero
    ];
}
```

NotebookLM

O Dilema e a Ferramenta

Mapeando Argumentos de Forma Explícita



Quando uma função possui muitos parâmetros de tipos semelhantes, a injeção baseada unicamente na posição exige adivinhação e facilita a inversão desastrosa de valores durante a invocação.

Solução: Argumentos Nomeados (Named Arguments). A linguagem permite injetar valores referenciando explicitamente a variável de destino, ignorando completamente a ordem original.

```
nomeFuncao(
    nomeParametroDesejado: $valor1,
    outroParametro: $valor2
);
```

NotebookLM

A Aplicação Prática

Mapeando Argumentos de Forma Explícita

Contexto: Invocando a fábrica de filmes com total clareza semântica, mesmo alterando a ordem original dos parâmetros.

```
$filme = criaFilme(
        nota: 7.8,
        genero: 'super-herói',
        anoLancamento: 2021,
        nome: 'Thor: Ragnarok'
);
```

Ordem original ignorada. Precisão garantida pela nomeação.

NotebookLM

O Dilema e a Ferramenta

Isolando Responsabilidades na Arquitetura

Deixar a declaração de lógicas (funções) e os scripts de execução aglomerados na mesma raiz gera uma base de código confusa, frágil e propensa a brechas de segurança no acesso via servidor.

Solução: Segregação arquitetural de diretórios. O código-fonte lógico é estritamente confinado (/src), enquanto a execução é orquestrada por um único ponto de entrada confiável.

```
// Ponto de Entrada (ex: index.php) requisitando lógicas isoladas
require __DIR__ . '/caminho_seguro/dependencia.php';
```

NotebookLM

A Aplicação Prática

Isolando Responsabilidades na Arquitetura

Contexto: Estruturando o projeto real: movendo o repositório de lógicas e criando o arquivo index.php.

```
/meu-projeto
├── src/
│   └── funcoes.php
├── filme.json
└── index.php
```

index.php

```php
// Única porta de entrada orquestrando as dependências protegidas
require __DIR__ . '/src/funcoes.php';

// A execução principal do catálogo ocorre aqui de forma segura...
```

NotebookLM